Leopold–Franzens–University
Innsbruck

Institute of Computer Science
Distributed and Parallel Systems

Bachelor Thesis

Supervisors:

Innsbruck, Austria
2 June 2026

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt durch meine eigenhändige Unterschrift, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder inhaltlich den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht. Ich erkläre mich mit der Archivierung der vorliegenden Bachelorarbeit einverstanden.

Datum	Unterschrift

Abstract

General considerations: Check English grammar (Grammarly plugin is highly recommended to deliver correct text). Writing should be formal throughout the whole thesis, and considering the scientific aspect of the Bachelor thesis, i.e., justify hard assumptions using proper references to scientific articles.

Contents

1 Introduction

The introduction should explain the addressed problem, summarize existing systems (related works), and identify the need for the proposed solution. Identify the contributions of the proposed system. It is always good to defend your points and arguments with references to papers.

2 Related Work

2.1 LLM Serving and Resource Management

LLM Serving systems try to optimize the way LLM inference tasks map to specific hardware resources. Current systems often separate and classify different tasks to further enhance this mapping. BROS [**?**] distinguishes between latency-critical (RT) requests and throughput-oriented (BE) requests. EconoServe [**?**] makes a similar distinction between prompt processing tasks (PT) and generation tasks (GT). An important aspect of optimization in these systems is advanced KV Cache (KVC) handling. BROS designs a bidirectional KVC management mechanism to best fit hybrid RT and BE requests, while EconoServe maintains separate queues for PT and GT requests and designs its own KVC pipelining method. Because LLM inference is dominated by decoding, the KVC - not raw compute - is what most often caps throughput, which explains why both systems invest so heavily in cache-level optimization.

Beyond memory, energy consumption has emerged as a critical serving constraint. DynamoLLM [**?**] introduces a broad energy-management framework that dynamically reconfigures the number of active instances, model parallelism, and GPU frequencies in response to fluctuating workloads. This allows the system to minimize power consumption and carbon emissions while strictly satisfying latency constraints. The framework decomposes its decisions into a hierarchy of controllers operating on different timescales, separating slow choices such as instance count from fast ones such as GPU frequency scaling.

Finally, systems like NexusSched [**?**] look at the layer above and try to bridge the "information gap" between the central cluster router and the individual inference engines. Instead of relying on naive metrics like queue length, NexusSched uses forward-looking telemetry - such as predicted latency and available KVC - to make proactive routing decisions across the cluster. The motivating insight is

that pending request count can be misleading on engines under KVC pressure, where a short queue still produces long latencies. NexusSched therefore replaces reactive load balancing with predictive scheduling that anticipates bottlenecks before they appear.

However, while these infrastructure-level serving systems excel in maximizing hardware utilization and energy efficiency, they generally treat prompts as generic computational blocks and lack deeper semantic awareness.

2.2 Prompt-Aware LLM Routing

While LLM serving asks the question "How?", LLM routing tries to optimize the answer to "Who?". Here, the choice between models of different sizes and specializations is made. This is evidently the most important factor to find the best trade-off between resource consumption and inference accuracy [**?** **?** **?**].

A brute-force approach to make this choice is presented by CARGO [**?**], which sends each prompt to multiple LLMs and grades the answers using separate "judge" LLMs. These results are then used to train a routing regression model used for deployment.

The most logical and the most studied approach is to make this choice by inspecting/analyzing the input prompt and route based on its features. TagRouter [**?**] uses custom semantic Tags to classify prompts and lets the historically best model for these tags do the inference. Marinelli et al. [**?**] suggest the new metric "Number of Thoughts" to describe the prompt's complexity. This single value is then predicted through a random forest and used for routing. The IRT-Router [**?**] scores responses across 25 predefined abilities and uses Item Response Theory to make the router improve over time.

Although using the input prompt as the only anchor for this choice might already seem like the best approach, there are still clear limitations to be noted [**?**]. Lookahead [**?**] introduces an additional lookahead routing framework that predicts potential model outputs without full inference. Thus, enabling more informative routing. GreenServ [**?**], the work this paper is largely based on, explicitly optimizes the tradeoff between accuracy and latency/energy. To achieve this, GreenServ extracts three features from each prompt: The Task Type, the Semantic Cluster, and the Text Complexity. The Task Type is used to

express the intent (e.g. is this a math problem, or a summarization task?). The Semantic Cluster groups the query by topic or domain. And the Text Complexity describes how complex the prompt itself is. After this, the reinforcement learning concept contextual multi-armed bandit is used to create a router that is learning continuously and online. The features of the prompt are fed into the router, the bandit selects a model and lastly updates its preferences based on the received reward.

While the approaches above thoroughly inspect and optimize routing decisions, they solely deploy their models in a single cloud environment. This makes them blind to the capabilities and possible benefits of edge computing and the computing continuum in general.

2.3 Dynamic Resource Allocation for LLM Serving

The systems discussed so far assume a fixed deployment: a static pool of models occupies a static pool of GPUs, and the scheduler only decides which already-deployed model serves each request. A separate body of work asks the orthogonal question of *how the deployment itself should change over time* - which models to replicate, on which devices, at what scale - in response to workload and hardware state. Recent surveys of LLM serving scheduling [**?**] highlight that this axis is increasingly inseparable from routing, since per-query decisions are bounded by capacity decisions made on a slower timescale.

ENOVA [**?**] addresses this for serverless multi-GPU clusters. It deconstructs the inference pipeline into measurable sub-stages, fits a recommendation module that maps each (LLM, GPU type) pair to an expected throughput configuration, and feeds live telemetry - request rate, pending count, GPU utilization, response time - into an autoscaling controller that adjusts how many replicas of each model are kept warm. The unit of allocation is the model itself, which is precisely the granularity at which the routing decisions in this thesis would benefit from elasticity.

SageServe [**?**] attacks the same problem at hyperscaler scale. Operating on production traces from Microsoft Office 365 (over ten million LLM requests per day across multiple regions), it co-optimizes short-term request routing with long-term GPU placement: an ARIMA traffic forecast predicts per-region demand and

an integer linear program reallocates GPU VMs and model placements every few hours, subject to SLA constraints. The authors report up to 25% GPU-hour savings against the production baseline and explicitly demonstrate that routing and placement decisions cannot be made in isolation without significant load mismatch.

A complementary line of work keeps the deployment fixed and instead reshapes the model itself to fit the device. CLONE [**?**] customizes a single LLM for a single edge endpoint by combining structured pruning, LoRA-based accuracy recovery, and layer-level dynamic voltage and frequency scaling to meet a latency target under a power budget. While not a cross-model allocation system, it is the natural counterpart on the edge tier: where ENOVA and SageServe ask which model lives on which GPU, CLONE asks how much capacity a fixed (model, GPU) pair can be made to yield.

The common control-loop pattern - ingest live telemetry, decide on a reconfiguration, dispatch - aligns directly with the telemetry surface this thesis already publishes (per-node queue length, per-model latency, free GPU memory). The present work treats the deployment as static and focuses on the per-query routing decision on top of it; integrating an allocation controller of this kind that consumes the same telemetry surface and updates the set of eligible nodes for each model between bandit decisions is a natural extension.

2.4 Research Gap and Thesis Contribution

While infrastructure-level serving systems excel at maximizing hardware utilization and managing memory, they treat prompts as generic computational blocks. In contrast, advanced semantic routers like GreenServ are highly intelligent in regard to prompt complexity, but operate under the assumption of infinite centralized cloud resources, remaining blind to physical network topologies and real-time hardware constraints. Currently, no system effectively combines deep semantic prompt analysis with strict, real-time physical resource limits across distributed environments. Dynamic per-model resource allocation, surveyed in the previous section, forms a complementary third axis that this thesis treats as out of scope but explicitly designs around: the telemetry surface introduced in Chapter **??** exposes exactly the per-node, per-model signals that such a controller would

consume.

This thesis bridges this gap by proposing a novel Collaborative Inference framework within the Edge-Cloud Continuum. By combining the strict, resource-aware telemetry of modern serving systems with the intelligent, prompt-aware multi-armed bandit routing of GreenServ, this work introduces a complete scheduling architecture.

The resulting system dynamically evaluates the real-time resource state of edge and cloud devices - filtering out nodes lacking the compute to meet strict latency deadlines - before analyzing the semantic complexity of the incoming prompt. It then dynamically selects the optimal execution location (distributing tasks between lightweight Edge models and powerful Cloud models). This ensures that inference accuracy is maximized for complex prompts while strictly adhering to real-time physical resource limits and energy efficiency goals.

Table 2.1: Detailed Summary of Related Work in LLM Serving and Routing

Paper	Focus Area	Goals (Acc, Lat, Res)	Detailed Methodology	Prompt Analysis Method
BROS [?]	Serving	No, Yes, Yes	Implements a bidirectional KVC allocation strategy that dynamically shares cache blocks between latency-critical and throughput-oriented requests.	None (Treats requests as generic computational loads).
EconoServe [?]	Serving	No, Yes, Yes	Separates prompt processing and generation into separate queues, utilizing a novel KVC pipelining technique to maximize GPU sharing.	None.
DynamoLLM [?]	Serving	No, Yes, Yes	Employs hierarchical controllers to dynamically scale active instances, model parallelism, and GPU frequencies based on traffic.	Workload profiling (Analyzes request lengths and arrival rates, but lacks semantic awareness).
NexusSched [?]	Serving (Cluster)	No, Yes, Yes	Uses forward-looking telemetry (predicted processing latency and available KVC) to make proactive routing decisions across distributed engines.	None.
CARGO [?]	Routing	Yes, No, Yes	Uses LLM-as-a-judge to score historical model answers, training a regression model to predict model confidence for incoming queries.	Encodes the input prompt into an embedding vector to feed into the routing regression model.
TagRouter [?]	Routing	Yes, No, Yes	Maps incoming queries to specific models that have historically performed best for the identified semantic tags.	Extracts custom semantic tags using a lightweight classifier prior to inference.
Marinelli et al. [?]	Routing	Yes, Yes, Yes	Uses a random forest classifier to predict required reasoning complexity, routing to a model sufficiently capable of handling the load.	Analyzes prompt metadata to estimate the required "Number of Thoughts" (reasoning complexity).
IRT-Router [?]	Routing	Yes, No, Yes	Applies Item Response Theory (IRT) to mathematically model both the latent abilities of 20+ LLMs and the difficulty of incoming prompts.	Estimates the latent difficulty of the prompt across 25 predefined cognitive abilities.
Lookahead [?]	Routing	Yes, No, No	Prevents blind routing by predicting potential model outputs (fast drafting) without full inference to gauge which model is best suited.	Employs fast-drafting/partial inference to preview the generation trajectory of the prompt.
GreenServ [?]	Routing	Yes, Yes, Yes	Utilizes a Contextual Multi-Armed Bandit (LinUCB) to continuously learn the optimal routing policy balancing accuracy and energy.	Extracts Task Type, Semantic Cluster (via embeddings), and Text Complexity (Flesch Reading Ease).
This work	**Routing & Serving**	**Yes, Yes, Yes**	**Two-stage pipeline: mathematically filters out nodes unable to meet hardware/latency constraints, then routes via a semantic LinUCB MAB.**	**Extracts Task Type, Semantic Cluster, and Text Complexity (adapted from GreenServ).**

3 System Model

The objective of this work is to dynamically orchestrate Large Language Model (LLM) inference tasks across an Edge-Cloud continuum. The system aims to maximize inference accuracy while minimizing physical energy consumption. Furthermore, real-time latency constraints dictated by the hardware's current operational state are considered.

3.1 Problem Formulation

Consider a distributed computing environment consisting of a set of available servers, comprising both resource-constrained edge nodes and high-capacity cloud nodes. Let $M = \{m_1, m_2, \ldots, m_K\}$ denote the heterogeneous pool of LLMs deployed across this continuum. A subset of smaller models $M_{edge} \subset M$ is deployed at the edge, while larger, high-precision models $M_{cloud} \subset M$ reside in the cloud.

The system processes a sequential stream of user queries over time. At each discrete time step t, a user submits a query q_t. Upon arrival, the system must select exactly one model $m_t \in M$ to execute the inference.

To ensure Quality of Service (QoS), the system must satisfy a strict latency deadline, T_{max}. The total latency $T(m_k, q_t)$ of executing query q_t on model m_k is the sum of a tier-dependent network transmission delay and the inference processing delay on the node hosting m_k. The inference delay depends on the real-time hardware state of that node, specifically its pending workload queue W_k and its available GPU memory M_k^{free}. The processing delay is estimated from the node's recent per-model latency history, so a node that is currently overloaded - or that has historically been slow for m_k - is reflected with a correspondingly larger $T(m_k, q_t)$. Nodes whose free GPU memory falls below a configured threshold are pruned outright (they cannot host the model at all), and the system then filters out any remaining models that cannot satisfy the deadline, establishing a

dynamic set of feasible models:

$$M_t^* = \{m \in M \mid T(m, q_t) \leq T_{max}\}$$

For the selected model $m_t \in M_t^*$, the system observes an inference accuracy $Acc(m_t, q_t)$ and a physical energy consumption $E(m_t, q_t)$. The routing problem is formulated as a multi-objective optimization task maximizing the scalarized reward r_t:

$$r_t = \alpha \cdot Acc(m_t, q_t) - \beta \cdot E(m_t, q_t)$$

where α and β are user-defined weighting parameters that specify the trade-off between output quality and energy efficiency. In practice, both terms are normalized to $[0, 1]$ before being combined, so the bandit always compares arms on the same scale regardless of the absolute units of accuracy and energy. Because the true accuracy and energy consumption are only observed for the selected model after execution, the system must continuously learn the optimal routing policy online.

3.2 Proposed Method

To solve the routing problem defined above, this thesis proposes a four-stage scheduling pipeline. The architecture bridges real-time hardware telemetry with semantic prompt analysis to achieve resource-aware and prompt-aware inference. The proposed architecture operates through the following sequential stages:

1. Telemetry and Feasibility Filtering: Continuous monitoring agents are deployed on all edge and cloud nodes. These agents asynchronously broadcast their real-time state - the pending workload queue W_k, the available GPU memory M_k^{free}, and a rolling per-model average inference latency - back to the central scheduler. To minimize unnecessary network transmission delays, the central scheduler is deployed as a gateway at the Edge tier. This allows it to be the initial point of contact and intercept incoming queries. When query q_t arrives, the scheduler first prunes any node whose free GPU memory falls below a configured threshold, then estimates the expected latency $T(m_k, q_t)$ for every model on its remaining nodes by combining a tier-dependent network delay with the queue-aware inference estimate $(W_k + 1) \cdot \bar{\tau}(m_k, k)$, where $\bar{\tau}(m_k, k)$ is node k's recent

average latency for model m_k. Any model whose best-case latency across the surviving nodes exceeds T_{max} is excluded, yielding the feasible set M_t^*. This stage guarantees that the system never routes a prompt to a node incapable of meeting the latency deadline, preventing bottlenecks.

2. Semantic Context Extraction: Simultaneously, the incoming query q_t is passed through a lightweight context generator. This module extracts key semantic features from the prompt to evaluate its complexity without performing full inference. Following the methodology established by GreenServ, the module extracts three specific features: Task Type (using a lightweight classifier), Semantic Cluster (via embedding-based topic clustering), and Text Complexity (via the Flesch Reading Ease formula). These categorical and numerical features are combined into a multi-dimensional context vector $x_t \in \mathbb{R}^d$.

3. Contextual Bandit Routing: With the feasible hardware set M_t^* and the semantic context vector x_t established, the system employs a Contextual Multi-Armed Bandit (MAB) algorithm to make the final routing decision. Specifically, the Linear Upper Confidence Bound (LinUCB) algorithm is utilized. The algorithm assumes a linear relationship between the context x_t and the expected reward of each model arm. LinUCB calculates the expected reward for all $m \in M_t^*$, balancing the use of historically successful models for similar prompts with the exploration of uncertain models. The model m_t that yields the highest upper confidence bound is selected, and the query is dispatched over the network to the corresponding edge or cloud node.

4. Execution and Policy Update (Feedback Loop): The selected node executes the inference task and returns the generated response to the user. Following execution, the system logs the actual physical energy consumed by the GPU $E(m_t, q_t)$ and computes the accuracy of the response $Acc(m_t, q_t)$ to calculate the final reward r_t. This scalar reward is fed back into the LinUCB agent, which updates its parameter matrices. This continuous feedback loop ensures that the scheduler adapts to distribution shifts in user queries and gracefully handles the dynamic integration or degradation of models within the computing continuum.

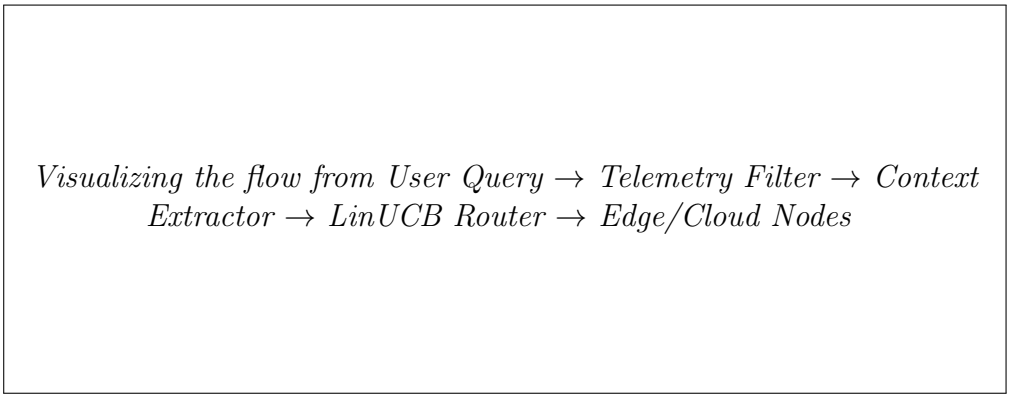


Figure 3.1: Overview of the Proposed Edge-Cloud Collaborative Scheduling Architecture.

4 Evaluation

The evaluation should start with a summary of the content of the section. It is always a good practice to give the reader some details about the system implemented and the important aspects that should be evaluated. Then, explain what the evaluation will look like. Explain the infrastructure used, metrics, competitors, or baselines.

Then, use subsections to explain more in detail each particular experiment or scenario and present results in a clear way, discussing every significant detail.

5 Conclusions and future work

Conclusions should highlight the problem solved and summarize the contributions done. Support this claiming with a few numeric results. Quickly discuss the limitations of the work and future work scope.

Please, find attached a checklist recommended by SIGPLAN that will be extremely valuable to follow during the evaluation of your work.

SIGPLAN Empirical Evaluation Checklist

*This checklist is meant to **support** informed judgement, not **supplant** it.*

Clearly Stated Claims — *Example Violations*

""

Claims not explicit
Claims must be explicit in order for the reader to assess whether the empirical evaluation supports them. Missing claims cannot possibly be assessed. Claims should also aim to state not just what is achieved but how.

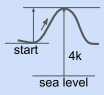
"precise analysis"

Claims not appropriately scoped
The truth of a claim should clearly follow from the evidence provided. Claims that are not fully supported mislead readers. 'Works for all Java' is over-broad when based on a subset of Java. Other examples are 'works on real hardware' when evaluating only with (unrealistic) simulation, and 'automatic process' when requiring human intervention.

"devs liked it"

Fails to acknowledge limitations
A paper should acknowledge its limitations to place the scope of its results in context. Stating no limitations at all, or only tangential ones, while omitting the more relevant ones may mislead the reader into drawing overly-strong conclusions. This could hold back efforts to publish future improvements, and may lead researchers down wrong paths.

Suitable Comparison — *Example Violations*



"hiked 4k mountain"

Fails to compare against appropriate baseline
Empirical evidence for a claim that a technique/system improves upon the state-of-the-art should include a comparison against an appropriate baseline. The lack of a baseline means empirical evidence lacks context. A 'straw man' baseline that is misrepresented as state-of-the-art is also problematic, as it would inflate apparent benefit.

"finished before you"

Comparison is unfair
Comparisons to a competing system should not unfairly disadvantage that system. Doing so would inflate the apparent advantage of the proposed system. For example, it would be unfair to compile the state-of-the-art baseline at -O0 optimization level, while using -O3 for the proposed system.

Principled Benchmark Choice — *Example Violations*

"low sync overhead"

Inappropriate suite
Evaluations should be conducted using appropriate established benchmarks where they exist so that claimed results are more likely to generalize. Not doing so may yield results that are not sufficiently general. Established suites should be used in context; e.g. it would be wrong to use a single-threaded suite for studying parallel performance.

"we used TheBench"

Unjustified use of non-standard suite(s)
The use of standard benchmark suites improves the comparability of results. However, sometimes a non-standard suite, such as one that is subsetted or homegrown, is the better choice. In that case, a rationale, and possible limitations, must be provided to demonstrate why using a standard suite would have been worse.

"large speedup"

Kernels instead of full applications
Kernels can be useful and appropriate in a broader evaluation. However, a claim that a system benefits applications should be tested on such applications directly, and not only on micro-kernels, which may lack important characteristics of full applications.

Adequate Data Analysis — *Example Violations*

"10 seconds"

Insufficient number of trials
Modern systems with non-deterministic performance properties may require many trials (e.g., of a single time measurement) to characterize their behavior adequately. Failure to do so risks treating noise as signal. Similarly, more trials may be needed to get the system into an intended state (e.g., into a steady state that avoids warm-up effects).

"mean of 50 seconds"

Inappropriate summary statistics
Summary statistics such as mean and median can usefully characterize many results. But they should be selected carefully, because each statistic presents an accurate view only under appropriate circumstances. An inappropriate summary may amplify noise or hide an important trend.

"50 seconds"

No data distribution reported
A measure of variability (e.g., variance, std. deviation, quantiles) and/or confidence intervals is needed to understand the distribution of the data. Reporting just a measure of central tendency (e.g., a mean or median) can mislead the reader, especially when the distribution is bimodal or has significant variance.

Relevant Metrics — *Example Violations*

"energy consumed"

Indirect or inappropriate proxy metric
Proxy metrics can substitute for direct ones only when the substitution is clearly, explicitly justified. For example, it would be misleading and incorrect to report a reduction in cache misses to claim actual end-to-end performance or energy consumption improvement.

"devs were satisfied"

Fails to measure all important Effects
All important effects should be measured to show the true cost of a system. For example, compiler optimizations may speed up programs at the cost of drastically increasing compile times of large systems, so the compile time should be measured as well as the program speedup. Failure to do so distorts the cost/benefit of the system.

Appropriate and Clear Experimental Design — *Example Violations*

"sped up apache"

Insufficient information to repeat
Experiments evaluating an idea need to be described in sufficient detail to be repeatable. All parameters (including default values) should be included, as well as all version numbers of software, and full details of hardware platforms. Insufficient information impedes repeatability and comparison of future ideas and can hinder scientific progress.

"for sensor net"

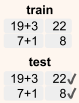
Unreasonable platform
The evaluation should be on a platform that can reasonably be said to match the claims; otherwise, the results of the evaluation will not fully support the claims. For example, a claim that relates to performance on mobile platforms should not have an evaluation performed exclusively on servers.

"10 times faster"

Ignores key design parameters
Key parameters should be explored over a range to evaluate sensitivity to their settings. Examples include the size of the heap when evaluating garbage collection and the size of caches when evaluating a locality optimization. All expected system configurations (e.g., from warmup to steady state) should be considered.

"prompt treatment"

Gated workload generator
Load generators for typical transaction-oriented systems should be 'open loop', to generate work independent of the performance of the system under test. Otherwise, results are likely to mislead because real-world transaction servers are usually open-loop.



"perfect"

Tested on training set
When a system aims to be general but was developed with close consideration of specific examples, it is essential that the evaluation explicitly perform cross-validation, so that the system is evaluated on data distinct from the training set. For example, a static analysis should not be exclusively evaluated on programs used to inform its development.

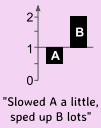
Appropriate Presentation of Results — *Example Violations*


"have up to 4 leaves"

Misleading summary of results
The summary of the results must reflect the full range of their character to avoid misleading the reader. For example, it is not appropriate to summarize speedups of 4%, 6%, 7%, and 49% as 'up to 49%'. Instead, the full distribution of results must be reported.

"B is much faster"

Inappropriately truncated axes
Graphs provide a visual intuition about a result. A truncated graph (with an axis not including zero) will exaggerate the importance of a difference. 'Zooming' in to the interesting range of an axis can sometimes aid exposition, but should be pointed out explicitly to avoid being misleading.



"Slowed A a little, sped up B lots"

Ratios plotted incorrectly
Incorrectly plotted ratios badly mislead visual intuition. For example, 2.0 and 0.5 are reciprocals, but their linear distance from 1.0 does not reflect that, so plotting those numbers on a linear scale significantly distorts the result. This misleading effect can be avoided either by using a log scale or by normalizing to the lowest (highest) value.


"9.36 s startup time"

Inappropriate level of precision
Measurements reported at a proper level of precision reveal relevant information. Under-precise reports may hide such information, and over-precise ones may overstate the accuracy of a measurement and obscure what is relevant. For example, reporting '49.9%' when the experimental error is +/- 1% overstates the level of precision of the result.

October 2018. E. D. Berger, S. M. Blackburn, M. Hauswirth, and M. Hicks for the ACM SIGPLAN EC

Notes

Claims not Explicit This includes *implied* generality — implied: *'works for all Java'*, but actually only on a static subset; implied: *'works on real hardware'*, but actually only works in simulation; implied: *'automatic process'*, but in fact required non-trivial human supervision; implied: *'only improves the systems' performance'*, but actually the approach requires breaking some of the system's expected behavior.

Fails to Acknowledge Limitations One concern we have heard multiple times is that this example, previously titled *Threats to validity*, is not useful. The given reason is that *threats to validity* sections in software engineering papers often mention threats of little significance while ignoring real threats. This is unfortunate, but does not eliminate the need to clearly scope claims, highlighting important limitations. For science to progress, we need to be honest about what we have achieved. Papers often make, or imply, overly strong claims. One way this is done is to ignore important limitations. But doing so discourages or undervalues subsequent work that overcomes those limitations because that progress is not appreciated. Progress comes in steps, rarely in leaps, and we need those steps to be solid and clearly defined.

Fails to Compare Against Appropriate Baseline The baseline could also be an unsophisticated approach to the same problem, e.g., a fancy testing tool is usefully compared against one that is purely random, in order to see whether it does better.

Inappropriate Suite This includes misuse of incorrect established suite e.g. use of SPEC CINT2006 when considering parallel workloads.

Unjustified Use of Non-Standard Suite(s) A concern we heard was that use of standard suites may lead to work that overfits to that benchmark. While this is a problem in theory, and is well known from the machine learning community, our experience is that PL work more often has the opposite problem. Papers we looked at often subset a benchmark, or cherry-picked particular programs. Doing so calls results into question generally, and makes it hard to compare related systems across papers. We make progress more clearly when we can measure it. Good benchmark suites are important, since only with them can we make generalizable progress. Developing them is something that our community should encourage.

Note that *'benchmark'* in this category includes what is measured and the parameters of that measurement. One example of an oft-unappreciated benchmark parameter is timeout choice.

Inappropriate Summary Statistics As particular best practices: The geometric mean should only be used when comparing values with different ranges, and the harmonic mean when comparing rates. When distributions have outliers, a median should be presented. There are many excellent resources available, including: *Common errors in statistics (and how to avoid them).* (Phillip I Good and James W Hardin, 2012), *What is a P-value anyway?: 34 stories to help you actually understand statistics.* (Andrew Vickers, 2010), and *Statistical misconceptions.* (Schuyler W Huck, 2009).

Ratios Plotted Incorrectly For example, if times for a and b are 4 sec and 8 sec respectively for benchmark x and 6 sec and 3 sec for benchmark y, this could be shown as a/b (0.5, 2.0) or b/a (2.0, 0.5), where 1.0 represents parity. Although the results (0.5 & 2.0) are reciprocals, their distance from 1.0 on a linear scale is different by a factor of two (0.5 & 1.0), overstating the speedup. This is why showing ratios (or percentages) greater than 1.0 (100%) and less than 1.0 (100%) on the same linear scale is visually misleading.

FAQ

Why a checklist? Our goal is to help ensure that current, accepted best practices are followed. Per the Checklist Manifesto, checklists help to do exactly this. Our interest is the good practices for carrying out empirical evaluations as part of PL research. While some practices are clearly wrong, many require careful consideration: Not every example under every category in the checklist applies to every evaluation – expert judgment is required. The checklist is meant to assist expert judgment, not substitute for it. 'Failure isn't due to ignorance. According to best-selling author Atul Gawande, it's because we haven't properly applied what we already know.' We've kept the list to a single page to make it easier to use and refer back to.

Why now? When best practices are not followed, there is a greater-than-necessary risk that the benefits reported by an empirical evaluation are illusory, which harms further progress and stunts industry adoption. The members of the committee have observed many recent cases in which practices in the present checklist are not followed. Our hope is that this effort will help focus the community on presenting the most appropriate evidence for a stated claim, where the form of this evidence is based on accepted norms.

Is use of the checklist going to be formally integrated into SIGPLAN conference review processes? There are no plans to do so, but in time, doing so may make sense.

How do you see authors using this checklist? We believe the most important use of the checklist is to assist authors in carrying out a meaningful empirical evaluation.

How do you see reviewers using this checklist? We also view the checklist as a way to remind reviewers of important elements of a good empirical evaluation, which they can take into account when carrying out their assessment. However, we emphasize that proper use of the checklist requires nuance. Just because a paper has every box checked doesn't mean it should be accepted. Conversely, a paper with one or two boxes unchecked may still merit acceptance. Even whether a box is checked or not may be subject to debate. The point is to organize a reviewer's thinking about an empirical evaluation to reduce the chances that an important aspect is overlooked. When a paper fails to check a box, it deserves some scrutiny in that category.

How did you determine which items to include? The committee examined a sampling of papers from the last several years of ASPLOS, ICFP, OOPSLA, PLDI, and POPL, and considered those that contained some form of empirical evaluation. We also considered past efforts examining empirical work (Gernot Heiser's "Systems Benchmarking Crimes", the "Pragmatic Guide to Assessing Empirical Evaluations", and the "Evaluate Collaboratory"). Through regular discussions over several months, we identified common patterns and anti-patterns, which we grouped into the present checklist. Note that we explicitly did not intend for the checklist to be exhaustive; rather, it reflects what appears to us to be common in PL empirical evaluations.

Why did you organize the checklist as a series of categories, each with several examples? The larger categories represent the general breadth of evaluations we saw, and the examples are intended to be helpful in being concrete, and common. For less common empirical evaluations, other examples may be relevant, even if not presented in the checklist explicitly. For example, for work studying human factors, the Adequate Data Analysis category might involve examples focusing on the use of statistical tests to relate outcomes in a control group to those in an experimental group. More on this kind of work below.

Why did you use checkboxes instead of something more nuanced, like a score? The boxes next to each item are not intended to require a binary "yes/no" decision. In our own use of the list, we have often marked entries as partially filling a box (e.g., with a dash to indicate a "middle" value) or by coloring it in (e.g., red for egregious violation, green for pass, yellow for something in the middle).

What about human factors or other areas that require empirical evaluation? PL research sometimes involves user studies, and these are different in character than, say, work that evaluates a new compiler optimization or test generation strategy. Because user studies are currently relatively infrequent in the papers we examined, we have not included them among the category examples. It may be that new, different examples are required for such studies, or that the present checklist will evolve to contain examples drawn from user studies. Nonetheless, the seven category items are broadly applicable and should be useful to authors of any empirical evaluation for a SIGPLAN conference.

How does the checklist relate to the artifact evaluation process? Artifact evaluation typically occurs after reviewing a paper, to check that the claims and evidence given in the paper match reality, in the artifact. The checklist is meant to be used by reviewers while judging the paper, and by authors when carrying out their research and writing their paper.

How will this checklist evolve over time? Our manifesto is: Usage should determine content. We welcome feedback from users of the checklist to indicate how frequently they use certain checklist items or how often papers reviewed adhere to them. We also welcome feedback pointing to papers that motivate the inclusion of new items. As the community increasingly adheres to the guidelines present in the checklist, the need for their inclusion may diminish. We also welcome feedback on presentation: please share points of confusion about individual items, so we can improve descriptions or organization.

Feedback via: `http://www.sigplan.org/Resources/EmpiricalEvaluation/`